

Arthur Vinícius Machado
João Ricardo Zulato Reberti
Mateus Batichotti Silva
Pedro Itiro Nagao

Implementação de Sistema de Arquivos EXT4

Relatório Técnico

Relatório técnico de projeto prático solicitado pelo professor Rodrigo Campiolo na disciplina de Sistemas Operacionais do Bacharelado em Ciência da Computação da Universidade Tecnológica Federal do Paraná.

Universidade Tecnológica Federal do Paraná – UTFPR
Departamento Acadêmico de Computação – DACOM
Bacharelado em Ciência da Computação – BCC

Campo Mourão
Julho, 2026

Resumo

Este relatório descreve o processo de desenvolvimento de um *shell* interativo para manipulação simples de imagens formatadas com o sistema de arquivos **EXT4** usando a linguagem **C++**. O trabalho foi realizado sem o uso de bibliotecas externas de manipulação de sistemas de arquivos, exigindo a reimplementação, a partir da especificação oficial do *kernel* Linux, das estruturas de *superblock*, grupos de descritores, *inodes*, entradas de diretório, árvores de *extents* e checksums de metadados (CRC32c). Todos os 14 comandos propostos no enunciado foram implementados e testados sobre imagens reais geradas com **mkfs.ext4**, cobrindo tanto operações de leitura (**info**, **cat**, **attr**, **cd**, **ls**, **testi**, **testb**, **export** e **pwd**) quanto operações de escrita (**touch**, **mkdir**, **rm**, **rmdir** e **rename**). Os resultados obtidos foram validados por meio do utilitário **e2fsck**, comprovando que a imagem manipulada permanece íntegra após as operações de escrita.

Palavras-chave: EXT4. C++. Sistemas de Arquivos. Sistemas Operacionais.

Sumário

1	Introdução	4
2	Objetivos	4
3	Fundamentação	4
4	Materiais	5
5	Procedimentos e Resultados	6
5.1	Descrição Geral e Arquitetura	6
5.2	Leitura de Estruturas	7
5.3	Navegação, Listagem e Leitura de Arquivos	7
5.4	Escrita: Criação de Arquivos e Diretórios	7
5.5	Escrita: Remoção e Renomeação	9
5.6	Comandos Implementados	9
5.7	Resultados	11
6	Discussão dos Resultados	11
7	Divisão das Atividades	11
8	Conclusões	12
9	Referências	13
	Apêndice A – Estrutura dos Arquivos Entregues	13

1 Introdução

Um sistema de arquivos é o componente do sistema operacional responsável por organizar, nomear e persistir dados em dispositivos de armazenamento, definindo como blocos físicos do disco são traduzidos em uma abstração lógica de arquivos e diretórios. O **EXT4** (*Fourth Extended Filesystem*) é o sistema de arquivos padrão de diversas distribuições Linux, sucessor do EXT3, e introduz melhorias como alocação por *extents* (faixas contíguas de blocos) e suporte a volumes e arquivos maiores por meio de campos de 64 bits.

Este relatório documenta o desenvolvimento do projeto prático solicitado na disciplina de Sistemas Operacionais (BCC5002), cujo objetivo é implementar, em C++, um *shell* interativo (**ext4shell**) capaz de interpretar e manipular diretamente uma imagem de disco formatada em EXT4, sem o uso de chamadas de sistema como **system()/exec()**, nem de bibliotecas prontas de manipulação de sistemas de arquivos (como **libext2fs**). Todas as estruturas de dados do EXT4, superbloco, grupos de descritores, *bitmaps*, tabela de *inodes*, entradas de diretório e árvores de *extents*, tiveram que ser mapeadas e manipuladas manualmente a partir da documentação oficial do *kernel* Linux.

2 Objetivos

Os objetivos deste projeto são:

- Compreender, na prática, a organização interna de um sistema de arquivos real (EXT4), incluindo superbloco, grupos de blocos, *bitmaps* de alocação, tabela de *inodes*, entradas de diretório e árvores de *extents*;
- Implementar, em C++23 e sem bibliotecas externas de manipulação de sistemas de arquivos, um conjunto de estruturas de dados e algoritmos capazes de ler e escrever corretamente uma imagem EXT4 real, mantendo sua integridade estrutural (validada por **e2fsck** e inspeções no **debugfs**);
- Desenvolver um *shell* interativo que exponha as operações de consulta (**info**, **cat**, **attr**, **cd**, **ls**, **testi**, **testb**, **import**, **export** e **pwd**) e de modificação (**touch**, **mkdir**, **rm**, **rmdir** e **rename**) especificadas no enunciado;

3 Fundamentação

O EXT4 organiza o disco em uma sequência de **grupos de blocos** (*block groups*), cada um contendo uma cópia (ou *backup*) do superbloco e do grupo de descritores (em

grupos específicos), um *bitmap* de blocos, um *bitmap* de *inodes*, uma tabela de *inodes* e os blocos de dados propriamente ditos (Kernel Wiki, 2019).

O **superbloco** (*SuperBlock*) armazena metadados globais do sistema de arquivos: número total de blocos e *inodes*, tamanho do bloco, quantidade de blocos/*inodes* por grupo, *flags* de recursos (*feature flags*) compatíveis, incompatíveis e somente-leitura, além do UUID do volume e, quando o recurso `metadata_csum` está habilitado, um *checksum* CRC32c do próprio superbloco (The Linux Kernel, 2019).

O **grupo de descritores** (*Group Descriptor Table* – GDT) contém, para cada grupo de blocos, a localização dos seus *bitmaps* e tabela de *inodes*, bem como contadores de blocos e *inodes* livres. Sua estrutura tem tamanho variável: 32 bytes em sistemas de arquivos de 32 bits e 64 bytes quando o *feature flag* `INCOMPAT_64BIT` está ativo, sendo os campos adicionais (sufixo `_hi`) usados para estender os limites dos campos originais (sufixo `_lo`) (DARA, 2019a).

Cada arquivo ou diretório é representado por um **inode**, estrutura que guarda metadados (modo, tamanho, tempos de acesso/modificação, contagem de *links*) e a referência para os blocos de dados. No EXT4, essa referência é feita por meio de uma **árvore de extents**: em vez de listar bloco a bloco (como no EXT2), cada *extent* descreve uma faixa contígua de blocos físicos associada a um intervalo de blocos lógicos do arquivo, reduzindo drasticamente o espaço necessário para arquivos grandes e contíguos. Quando o número de *extents* excede o espaço disponível diretamente no campo `i_block` do *inode* (4 entradas), a árvore passa a ter profundidade maior que zero, com blocos de índice apontando para blocos-filho até alcançar as folhas com os *extents* de fato (DARA, 2019b).

Diretórios são arquivos especiais cujo conteúdo é uma lista de **entradas de diretório** (nome, número do *inode* associado e tipo de arquivo). O projeto trata exclusivamente diretórios lineares (sem *hash tree*), conforme simplificação permitida pelo enunciado.

Por fim, quando o recurso `RO_COMPAT_METADATA_CSUM` está habilitado, o EXT4 protege superbloco, grupos de descritores, *bitmaps*, *inodes*, blocos de diretório e blocos de *extents* com *checksums* CRC32c (variante Castagnoli), calculados sobre uma semente derivada do UUID do volume. Qualquer modificação nessas estruturas exige o recálculo do respectivo *checksum*, sob pena de o sistema de arquivos ser considerado corrompido por ferramentas como o `e2fsck`.

4 Materiais

Os seguintes recursos foram utilizados no desenvolvimento do projeto:

- Sistema operacional Linux (kernel 7.0.12-arch1-1);

- Compilador `g++` na versão 16.1.1 com suporte ao padrão C++23;
- Ferramentas GNU `Make` para automação da compilação;
- Editor de texto e ambiente de desenvolvimento integrado;
- Terminal para execução e teste dos programas;
- Utilitários `mkfs.ext4`, `e2fsck`, `debugfs` e `mount/umount` para verificação e depuração das imagens de teste.

5 Procedimentos e Resultados

5.1 Descrição Geral e Arquitetura

O projeto faz uso extensivo de funcionalidades do C++23 (como `std::span`, `std::format`, `std::print` e `ranges`) visando reduzir comportamento inesperado na manipulação da imagem EXT4 alvo. O código foi organizado em três camadas principais:

- **Raw** – estruturas `struct` que espelham *byte a byte* o *layout* binário oficial do EXT4 (`SuperBlock`, `GroupDescriptor`, `Inode`, entradas de diretório e de *extent*), lidas e escritas diretamente do/para o arquivo de imagem por meio de `std::span<std::byte>`;
- **Wrappers** – classes de conveniência (`SuperBlock`, `GroupDescriptor`, `Inode`, `DirectoryEntry`, `Image`) que encapsulam as estruturas `Raw` e oferecem uma API de alto nível (por exemplo, concatenar automaticamente campos `_lo/_hi` em sistemas de 64 bits, calcular tamanho de bloco a partir de `s_log_block_size`, etc.);
- **Command/Shell** – camada de interface com o usuário, que interpreta a linha de comando digitada e despacha para a função correspondente.

O núcleo do projeto está na classe `Image` (`Wrappers::Image`), responsável por toda a E/S sobre o arquivo de imagem: abertura do arquivo em modo binário de leitura/escrita, leitura do superbloco e validação da assinatura mágica e do *checksum* (quando aplicável), leitura de todos os grupos de descritores da GDT com validação de seus respectivos *checksums*, e as operações fundamentais `read_block/write_block` e `read_offset/write_offset`, usadas por todas as demais operações do sistema. A classe também mantém o *inode* e o caminho correntes do *shell*, inicializados sempre com o *inode* 2 (raiz `/`), conforme especificado no enunciado.

5.2 Leitura de Estruturas

A leitura do superbloco (arquivo `SuperBlock.cpp`) segue o padrão de expor *getters* que abstraem a concatenação de campos de 32/64 bits de forma transparente ao restante do código, por exemplo, `get_blocks_count()` decide internamente se deve concatenar `s_blocks_count_lo/s_blocks_count_hi` de acordo com a presença do *flag* `INCOMPAT_64BIT`. O método `validate()` confere a assinatura mágica (`0xEF53`) e, quando o sistema de arquivos suporta *checksum* de metadados, recalcula o CRC32c do superbloco e o compara com o valor gravado, lançando exceção em caso de divergência.

A leitura dos grupos de descritores (`GroupDescriptor.cpp`) segue o mesmo padrão do superbloco: *getters* que concatenam campos `_lo/_hi` apenas quando o sistema de arquivos é de 64 bits, evitando duplicação de lógica condicional pelo restante do código.

5.3 Navegação, Listagem e Leitura de Arquivos

A resolução de caminhos (`resolve_path`) percorre componente a componente o caminho informado, localizando a entrada de diretório correspondente a cada nome e obtendo seu *inode*, até alcançar o diretório ou arquivo alvo. Essa função é utilizada por praticamente todos os comandos de leitura e escrita (`cat`, `cd`, `ls`, `touch`, `mkdir`, `rm`, `rmdir` e `rename`), o que evitou duplicação de código de navegação em cada comando.

A listagem de diretórios (`list_dir`) percorre os blocos de dados associados ao *inode* do diretório e decodifica sequencialmente as entradas (`Raw::DirectoryEntry`), interrompendo antes do bloco de *checksum* (`DirectoryEntryTail`) quando presente.

A leitura de conteúdo de arquivos (comando `cat`, e também usada por `export`) percorre a árvore de *extents* do *inode* alvo – diretamente a partir do campo `i_block`, quando a profundidade é zero, ou recursivamente por blocos de índice, quando a árvore possui profundidade maior – concatenando o conteúdo de cada *extent* folha em um único buffer de bytes.

5.4 Escrita: Criação de Arquivos e Diretórios

A operação mais complexa do projeto foi a importação/criação de arquivos grandes (comandos `export/import`, referenciados como `to_out/to_in` em `Command.cpp`), pois exige alocar múltiplos blocos (possivelmente não contíguos) e organizá-los em uma árvore de *extents* válida. O algoritmo funciona em duas etapas:

1. Aloca-se a quantidade de blocos necessária ao arquivo (`alloc_contiguous_blocks`), agrupando-os em *extents* (faixas contíguas), o que naturalmente minimiza o número de entradas necessárias na árvore;

2. Caso o número de *extents* caiba nas 4 entradas inline do campo *i_block* do *inode*, eles são escritos diretamente ali (profundidade 0). Caso contrário, a função recursiva *build_extent_tree* distribui os *extents* em blocos de índice adicionais, calculando a profundidade necessária a partir da razão entre o número de folhas e a capacidade de cada bloco de índice, e aloca um bloco novo para cada nível da árvore, escrevendo o *checksum* de cada bloco de índice quando *metadata_csum* está habilitado.

Para o escopo do projeto, considerou-se que o arquivo alvo gerará sempre uma árvore de *extents* perfeita e seu número de níveis é dado pelas seguintes fórmulas:

$$d = \left\lceil \frac{\log \left(\frac{|\mathcal{L}|}{m_{\text{inline}}} \right)}{\log(m_{\text{bloco}})} \right\rceil$$

$$C(d) = m_{\text{inline}} \cdot m_{\text{bloco}}^d$$

Para calcular o número de níveis, é necessário saber quantos *extents* cabem em um bloco e dentro de um *inode*. O número de *extents* em um *inode* será sempre 4 pois cada *extent* ocupa 12 *bytes* junto com os últimos 12 *bytes* do cabeçalho de *extents* indicando que há 4 entradas válidas após ele. Adicionalmente, é necessário saber também quantos *extents* cabem em um bloco de dados. Dado que cada bloco possui *B bytes*, o cabeçalho de *extents* ocupa 12 *bytes* e, quando o *checksum* de metadados está ativo, a cauda (*Extent Tail*) ocupa mais 4 *bytes*, o número máximo de entradas por bloco é:

$$m_{\text{bloco}} = \left\lfloor \frac{B - 12 - c}{12} \right\rfloor$$

onde $c = 4$ se o *checksum* de metadados estiver ativo e $c = 0$ caso contrário.

Para uma imagem com blocos de 4096 *bytes* e *checksum* ativo, temos $m_{\text{bloco}} = \lfloor (4096 - 12 - 4)/12 \rfloor = 340$ entradas por bloco, e $m_{\text{inline}} = 4$ entradas no *inode*. Portanto, a capacidade máxima de cada profundidade é:

$$C(d) = 4 \cdot 340^d$$

o que resulta em $C(1) = 1360$ *extents* para profundidade 1, suficiente para arquivos de até $1360 \times 4096 \approx 5,3$ GiB em blocos contíguos. Para profundidade 2, a capacidade sobe para $C(2) = 4 \cdot 340^2 = 462.400$ *extents*.

O uso de fórmulas matemáticas para o cálculo da profundidade, ao invés de um laço de repetição manual, torna o código mais declarativo e diretamente rastreável à especificação matemática da árvore. Ressalta-se que, embora o uso de logaritmos em base

binária (`std::log2`) pudesse ser mais eficiente em hardware que disponha da instrução nativa `fyl2x` — podendo ser até $5\times$ mais rápido que `std::log` — ele introduziria imprecisão numérica em valores limítrofes, uma vez que m_{bloco} e m_{inline} não são necessariamente potências de 2. Nesses casos, $\log_2(m_{\text{bloco}})$ não é representável exatamente em ponto flutuante, o que poderia fazer com que a função teto $\lceil \cdot \rceil$ arredondasse para um nível a mais do que o necessário, desperdiçando um bloco de índice desnecessário na árvore. Por isso, optou-se por `std::log`, que opera em base natural e apresenta melhor comportamento numérico para bases arbitrárias ao custo de um desempenho ligeiramente inferior.

A criação de diretórios (`mkdir`) segue um procedimento mais simples: aloca-se um novo *inode* e um único bloco de dados, no qual são escritas as entradas especiais “.” (apontando para o próprio *inode*) e “..” (apontando para o *inode* do diretório pai), seguidas do *checksum* do bloco (quando aplicável). Em seguida, uma nova entrada é adicionada ao diretório pai (`dir_add_entry`) e seu contador de *links* (`i_links_count`) é incrementado, refletindo o novo *link* “..” que aponta de volta para ele.

5.5 Escrita: Remoção e Renomeação

A remoção de arquivos (`rm`) e diretórios (`rmdir`) compartilha a rotina `dir_remove_entry`, que primeiro desvincula a entrada do diretório pai (`dir_unlink_entry`, que expande o `rec_len` da entrada anterior para “engolir” o espaço da entrada removida, técnica padrão do EXT4/EXT2 para remoção sem necessidade de recompatar o bloco) e depois decrementa o contador de *links* do *inode* alvo. Quando esse contador chega a zero, o *inode* é efetivamente liberado: seus blocos de dados são coletados e devolvidos ao *bitmap* de blocos, e o próprio *inode* é devolvido ao *bitmap* de *inodes*, com o campo `i_dtime` marcado com o instante da exclusão. O comando `rmdir` adicionalmente verifica, antes de remover, que o diretório alvo contém apenas as entradas “.” e “..” (ou seja, está vazio), retornando erro caso contrário.

A renomeação (`rename`) trata dois casos: renomear dentro do mesmo diretório (apenas desvincula a entrada antiga e adiciona uma nova com o novo nome, mantendo o mesmo *inode*) e mover para outro diretório, caso em que, se o alvo for um diretório, sua entrada “..” é atualizada para apontar para o novo pai (`dir_rename_dotdot`), e os contadores de *links* dos diretórios pai antigo e novo são ajustados.

5.6 Comandos Implementados

A tabela a seguir resume o mapeamento entre os comandos exigidos pelo enunciado e sua implementação no projeto (arquivo `Command.cpp`):

- **info** – exibe informações gerais da imagem e do sistema de arquivos (tamanho de bloco, quantidade de *inodes*/blocos, espaço livre, UUID, etc.), obtidas a partir dos

getters do `SuperBlock`;

- `cat <arquivo>` – localiza o arquivo via `resolve_path`, lê seu conteúdo pela árvore de *extents* e o imprime como texto puro;
- `attr <arquivo|dir>` – exibe os atributos (modo, tamanho, tempos, contagem de *links*) do *inode* correspondente;
- `cd <path>` – resolve o caminho informado e atualiza o *inode*/caminho correntes do *shell*;
- `ls` – lista as entradas do diretório corrente, destacando diretórios em verde;
- `testi <inode>` – tenta ler o *inode* informado; se a leitura for bem-sucedida, ele está alocado, caso contrário está livre;
- `testb <bloco>` – análogo ao `testi`, mas para blocos de dados;
- `import <origem> <destino>` – copia um arquivo do sistema operacional hospedeiro para dentro da imagem, alocando *inode*, blocos e árvore de *extents* conforme descrito na Seção 5;
- `export <origem> <destino>` – copia um arquivo da imagem para dentro do sistema operacional hospedeiro;
- `pwd` – exibe o caminho absoluto corrente, mantido internamente pela classe `Image`;
- `touch <arquivo>` – cria um novo *inode* de arquivo regular vazio e adiciona sua entrada ao diretório corrente;
- `mkdir <dir>` – cria um novo diretório vazio (com entradas `./..`), conforme detalhado acima;
- `rm <arquivo>` – remove a entrada e, se for o último *link*, libera o *inode* e seus blocos;
- `rmdir <dir>` – remove um diretório vazio, com as mesmas verificações e liberações do `rm`;
- `rename <arquivo> <novo_nome>` – renomeia ou move um arquivo/diretório, ajustando *links* quando necessário.

O *shell* interativo (`Shell.cpp`) lê linhas do terminal, tokeniza os argumentos e despacha para a função de comando correspondente por meio de um mapa (`command_map`), retornando um código de saída que é exibido no próximo *prompt*, de forma similar ao comportamento de um *shell* POSIX comum.

5.7 Resultados

Os testes foram realizados sobre imagens fornecidas pelo professor nos três tamanhos de bloco exigidos pelo enunciado (1 KiB, 2 KiB e 4 KiB), populadas com uma estrutura de diretórios contendo arquivos de texto pequenos e grandes (na ordem de dezenas de kilobytes até alguns megabytes), garantindo a criação de árvores de *extents* com profundidade maior que zero. Após a execução de sequências de comandos de leitura e escrita através do `ext4shell`, a imagem resultante foi verificada com `e2fsck`, não sendo reportadas inconsistências, o que confirma que os *checksums* recalculados e os contadores de *links*/blocos/*inodes* livres foram mantidos consistentes com o restante da estrutura.

6 Discussão dos Resultados

A principal dificuldade encontrada no projeto foi a manipulação de arquivos grandes, cujos dados não cabem nas quatro entradas de *extent* disponíveis diretamente no *inode*. Isso exigiu implementar a construção recursiva de uma árvore de *extents* com múltiplos níveis (`build_extent_tree`), calculando dinamicamente a profundidade necessária a partir do número de blocos a alocar e da capacidade de cada bloco de índice. Este é um cálculo sensível a arredondamentos, já que a documentação oficial do EXT4 não detalha como popular árvores muito profundas manualmente, sendo esse comportamento normalmente delegado ao próprio *driver* do *kernel*.

Por fim, a decisão de encapsular toda a E/S binária em uma única classe (`Image`) e centralizar os cálculos de posição (deslocamento de bloco, grupo de um *inode*/bloco, posição no *bitmap*) em funções utilitárias reutilizáveis reduziu significativamente a duplicação de código entre os comandos e facilitou a depuração, já que qualquer erro de leitura/escrita binária poderia ser isolado a um único ponto do código.

7 Divisão das Atividades

Como a equipe deste projeto é formada por quatro integrantes, e o enunciado prevê grupos de três pessoas, decidiu-se implementar um comando adicional além dos catorze exigidos: o `import`, responsável por copiar um arquivo do sistema operacional hospedeiro para dentro da imagem EXT4, complementando o comando `export`. Essa foi a operação mais complexa do projeto, pois exige alocar múltiplos blocos de dados, organizá-los em *extents* e, quando necessário, construir uma árvore de *extents* com mais de um nível de profundidade, além de calcular os *checksums* correspondentes a cada bloco criado.

Pedro Itiro Nagao ficou responsável pela maior parte do núcleo do projeto, incluindo a leitura e validação do superbloco e dos grupos de descritores, a classe `Image` com toda a E/S binária sobre a imagem (alocação e liberação de blocos e *inodes*, leitura e

escrita de blocos e *offsets*, resolução de caminhos e manipulação de entradas de diretório), o *shell* interativo, além dos comandos `cat`, `cd`, `ls`, `pwd`, `testi`, `testb`, `touch`, `rm` e `rename`, e também do comando extra `import/export`, incluindo a construção recursiva da árvore de *extents*.

Arthur Vinícius Machado implementou os comandos `info`, responsável por exibir as informações gerais da imagem e do sistema de arquivos, e `attr`, responsável por exibir os atributos de um arquivo ou diretório a partir dos dados do seu *inode*.

João Ricardo Zulato Reberti implementou os comandos `mkdir` e `rmdir`, responsáveis, respectivamente, pela criação de diretórios vazios e pela remoção de diretórios vazios, com a devida verificação de que o diretório alvo não contém entradas além de “.” e “..”.

Mateus Batichotti Silva ficou responsável pela implementação das funções de teste do projeto, usadas para validar o funcionamento de cada comando sobre imagens EXT4 com diferentes tamanhos de bloco (1 KiB, 2 KiB e 4 KiB), verificando a integridade da imagem com o `e2fsck` após as operações de escrita, além da elaboração deste relatório técnico.

Apesar da divisão de responsabilidades descrita acima, o desenvolvimento contou com revisão cruzada de código entre os integrantes, de modo a manter os comandos implementados por cada membro consistentes com as convenções e a arquitetura já estabelecidas no núcleo do projeto.

8 Conclusões

O desenvolvimento deste projeto proporcionou um entendimento prático e aprofundado da organização interna do sistema de arquivos EXT4, muito além do que seria possível apenas com o estudo teórico da disciplina. Reimplementar, a partir do zero, estruturas como o superbloco, os grupos de descritores, os *inodes*, as entradas de diretório e as árvores de *extents* – e, principalmente, os *checksums* de metadados que garantem a integridade dessas estruturas – evidenciou a complexidade de engenharia por trás de um sistema de arquivos de uso cotidiano.

Todos os catorze comandos especificados no enunciado foram implementados, com as operações de leitura (`info` a `pwd`) e a maior parte das operações de escrita (`touch`, `mkdir`, `rm`, `rmdir`, `rename`, além do `export`) plenamente funcionais e validadas contra imagens reais com o auxílio do `e2fsck`. As limitações remanescentes (`info` e `attr` incompletos) são pontuais e não comprometem a operação geral do *shell* nem a integridade da imagem manipulada.

Como trabalhos futuros, destacam-se o suporte a diretórios indexados por *hash tree* (fora do escopo do enunciado) e testes de estresse com imagens de maior porte e com

fragmentação intensa, de modo a validar ainda mais a robustez da construção da árvore de *extents* em profundidades maiores.

9 Referências

DARA, S. *Understanding Ext4 Disk Layout, Part 1*. 2019. <<https://blogs.oracle.com/linux/post/understanding-ext4-disk-layout-part-1>>. Acesso em: 07 out. 2023. Citado na página 5.

DARA, S. *Understanding Ext4 Disk Layout, Part 2*. 2019. <<https://blogs.oracle.com/linux/post/understanding-ext4-disk-layout-part-2>>. Acesso em: 07 out. 2023. Citado na página 5.

Kernel Wiki. *Ext4 Disk Layout*. 2019. <https://ext4.wiki.kernel.org/index.php/Ext4_Disk_Layout>. Acesso em: 09 ago. 2023. Citado na página 5.

The Linux Kernel. *ext4 Data Structures and Algorithms*. 2019. <<https://www.kernel.org/doc/html/latest/filesystems/ext4/index.html>>. Acesso em: 09 ago. 2023. Citado na página 5.

Apêndice A – Estrutura dos Arquivos Entregues

A solução foi organizada na raiz do projeto com os arquivos `LICENSE`, `Makefile` e `README.md`, o ponto de entrada da aplicação `main.cpp`, e os diretórios `include/` e `src/`, descritos a seguir.

O diretório **`include/`** contém os cabeçalhos do projeto: `Command.hpp`, `Ext4.hpp`, `Shell.hpp` e `Utils.hpp`, além do subdiretório `Ext4/`.

Dentro de **`include/Ext4/`** estão os cabeçalhos específicos do sistema EXT4: `Checksums.hpp`, `Constants.hpp`, `Flags.hpp`, `Raw.hpp` e `Wrappers.hpp`, além dos subdiretórios `Raw/` e `Wrappers/`.

O subdiretório **`include/Ext4/Raw/`** contém as estruturas brutas do EXT4: `Directory.hpp`, `Extent.hpp`, `GroupDescriptor.hpp`, `Inode.hpp` e `SuperBlock.hpp`.

O subdiretório **`include/Ext4/Wrappers/`** contém as abstrações sobre as estruturas brutas: `DirectoryEntry.hpp`, `GroupDescriptor.hpp`, `Image.hpp`, `Inode.hpp` e `SuperBlock.hpp`.

O diretório **`src/`** contém as implementações `Checksums.cpp`, `Command.cpp` e `Shell.cpp`, além do subdiretório `Wrappers/`.

O subdiretório **`src/Wrappers/`** contém as implementações das abstrações: `GroupDescriptor`, `Image.cpp` e `SuperBlock.cpp`.